

Self-Learning: Topological Sorting — Data Structures

Definition: Topological Sorting of a **Directed Acyclic Graph (DAG)** is a linear ordering of vertices such that for every directed edge (u, v) , vertex u comes strictly **before** vertex v in the ordering.

1. Detailed Technical Explanation

Sample DAG (Task Dependencies):

```
( 5 ) ----> ( 0 ) <---- ( 4 )
    |           |
    v           v
( 2 ) ----> ( 3 ) ----> ( 1 )
```

Valid Topological Orders:

- $5 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$
- $4 \rightarrow 5 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 0$
- $5 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1 \rightarrow 0$

Key Conditions:

- Topological sorting is defined **ONLY for DAGs** (Directed Acyclic Graphs).
- If a graph contains a **cycle**, topological sorting is **IMPOSSIBLE** because circular dependencies cannot be resolved linearly.

2. Algorithms for Topological Sorting

1. Kahn's Algorithm (BFS In-Degree Approach)

- Compute the **In-Degree** of every vertex in the graph.
- Push all vertices with **In-Degree = 0** into a Queue.
- While Queue is not empty:
 - Dequeue front vertex u , append u to topological order.
 - For each neighbor v of u : Decrement $\text{in_degree}[v]--$.
 - If $\text{in_degree}[v] = 0$: Enqueue v .
- If output order length $< |V|$, the graph contains a **Cycle**!

2. DFS with Stack Approach

- Initialize a `visited[]` boolean array.
- For every unvisited vertex u , call recursive `topoDFS(u)`.
- In `topoDFS(u)`:
 - Mark u as visited.
 - Recursively call `topoDFS(v)` for all unvisited neighbors v .
 - After visiting all children, **push u onto a Stack**.

4. Pop elements from the Stack to obtain the topological ordering.

3. Real-World Applications

- **Build Systems (Make, Gradle, Webpack):** Resolving source code compilation dependencies.
 - **Task Scheduling:** Determining valid execution order for tasks with prerequisites.
 - **Package Managers (apt, pip, npm):** Installing package dependency chains.
 - **Course Prerequisite Planning:** Academic curriculum scheduling.
-

4. Quick Recall Flow

DAG Only → In-Degree = 0 Enqueue → Decrement Neighbors → Repeat → Linear Dependency Order
(Kahn's Algorithm)